

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Práctica: API REST para conversión de temperaturas con Spring Boot

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

Boleta: 2019630177

id634296@gmail.com

7CM2

19 de septiembre de 2026

Índice

1. Introducción	2
2. Desarrollo de la API	2
2.1. Descripción de la API	2
2.2. Tecnologías utilizadas	3
2.3. Estructura del proyecto y ejecución	3
2.4. Implementación	3
2.5. Pruebas con curl	4
2.5.1. Celsius a Fahrenheit	4
2.5.2. Celsius a Kelvin	5
2.5.3. Fahrenheit a Celsius	5
2.5.4. Fahrenheit a Kelvin	6
2.5.5. Kelvin a Celsius	6
2.5.6. Kelvin a Fahrenheit	6
2.6. Validación de datos	7
2.6.1. Valor no numérico	7
2.6.2. Parámetro faltante	7
2.6.3. Valor fuera de rango: por debajo del cero absoluto en Celsius	8
2.6.4. Valor fuera de rango: Kelvin negativo	8
2.6.5. Valor no finito: NaN	8
3. Conclusiones	9
4. Referencias Bibliográficas	9

Índice de figuras

1.	Celsius a Fahrenheit con <code>valor=10</code>	5
2.	Celsius a Kelvin con <code>valor=20</code>	5
3.	Fahrenheit a Celsius con <code>valor=212</code>	5
4.	Fahrenheit a Kelvin con <code>valor=32</code>	6
5.	Kelvin a Celsius con <code>valor=300</code>	6
6.	Kelvin a Fahrenheit con <code>valor=300</code>	6
7.	Valor no numérico: <code>valor=abc</code>	7
8.	Petición sin el parámetro <code>valor</code>	7
9.	Valor por debajo del cero absoluto en Celsius	8
10.	Kelvin negativo, por debajo de su mínimo	8
11.	Valor <code>NaN</code>	9

Índice de tablas

1.	Endpoints disponibles (todos GET con el parámetro ?valor=)	2
2.	Códigos de respuesta que devuelve la API	2
3.	Tecnologías del proyecto	3

1. Introducción

Esta práctica construye una API REST que convierte temperaturas entre las escalas Celsius, Fahrenheit y Kelvin. Está hecha con Spring Boot y expone seis endpoints **GET**, uno por cada par de escalas. Cada endpoint recibe el valor por query string, valida que sea un número físicamente posible y devuelve una respuesta JSON estructurada; cuando el dato no es válido, devuelve un error JSON con el código HTTP que corresponde. Los endpoints se probaron con `curl` contra la API en ejecución.

2. Desarrollo de la API

2.1. Descripción de la API

La URL base es `http://localhost:8082/api/temperatura`. El puerto 8082 se fija en `application.properties` con `server.port=8082`; sin esa línea, el Tomcat embebido usaría el 8080. Todos los endpoints usan el verbo **GET**, porque solo consultan un resultado y no crean ni modifican nada, y reciben un único parámetro obligatorio, `valor`, con la temperatura que se quiere convertir. La tabla 1 lista los seis endpoints y la fórmula que aplica cada uno.

Endpoint	Conversión	Fórmula
<code>/convertir-celsius-a-fahrenheit</code>	$^{\circ}\text{C} \rightarrow ^{\circ}\text{F}$	$F = C \times \frac{9}{5} + 32$
<code>/convertir-celsius-a-kelvin</code>	$^{\circ}\text{C} \rightarrow \text{K}$	$K = C + 273,15$
<code>/convertir-fahrenheit-a-celsius</code>	$^{\circ}\text{F} \rightarrow ^{\circ}\text{C}$	$C = (F - 32) \times \frac{5}{9}$
<code>/convertir-fahrenheit-a-kelvin</code>	$^{\circ}\text{F} \rightarrow \text{K}$	$K = (F - 32) \times \frac{5}{9} + 273,15$
<code>/convertir-kelvin-a-celsius</code>	$\text{K} \rightarrow ^{\circ}\text{C}$	$C = K - 273,15$
<code>/convertir-kelvin-a-fahrenheit</code>	$\text{K} \rightarrow ^{\circ}\text{F}$	$F = (K - 273,15) \times \frac{9}{5} + 32$

Tabla 1: Endpoints disponibles (todos **GET** con el parámetro `?valor=`)

Los resultados se redondean a dos decimales. La respuesta correcta es un objeto JSON con cuatro campos: el valor recibido, su unidad, el valor convertido y la unidad de destino.

```
{
  "valorOriginal": 10.0,
  "unidadOriginal": "°C",
  "valorRespuesta": 50.0,
  "unidadRespuesta": "°F"
}
```

Cuando la petición no es válida, la API devuelve otro objeto JSON con el código HTTP, su nombre, un mensaje legible y la ruta que se pidió. La tabla 2 resume los códigos que usa [1].

Código	Nombre	Cuándo se devuelve
200	OK	La conversión se calculó correctamente.
400	Bad Request	Falta <code>valor</code> , o no es un número (por ejemplo <code>abc</code> o <code>20,5</code>).
422	Unprocessable Content	El valor es un número, pero está por debajo del cero absoluto, o es NaN o infinito.

Tabla 2: Códigos de respuesta que devuelve la API

El límite de cada escala es el cero absoluto [2]: $-273,15^{\circ}\text{C}$, $-459,67^{\circ}\text{F}$ y 0K . Un valor exactamente igual al límite se acepta; uno menor se rechaza.

2.2. Tecnologías utilizadas

Tecnología	Uso
Java 21 (Eclipse Temurin 21.0.12)	Lenguaje. Se usa record para los DTO [3].
Spring Boot 4.1.1	Framework; configura el servidor y la aplicación [4].
Spring Web MVC (<code>spring-boot-starter-webmvc</code>)	<code>@RestController</code> , <code>@GetMapping</code> , <code>@RequestParam</code> .
Tomcat 11 embebido	Servidor; por eso el empaquetado es un JAR ejecutable.
Spring Boot DevTools	Reinicio automático durante el desarrollo.
Maven (<code>mvnw</code>)	Construcción y ejecución de pruebas.
JUnit 5	Pruebas unitarias del servicio de conversión.
<code>curl</code>	Cliente para probar los endpoints desde la terminal.

Tabla 3: Tecnologías del proyecto

El proyecto se generó con Spring Initializr (Maven, Java, Group `com.ipn.mx`, Artifact `apisimple`, empaquetado JAR) con las dependencias Spring Web y DevTools. No usa base de datos ni librerías externas: la conversión es aritmética pura.

2.3. Estructura del proyecto y ejecución

```
ApiSimple/  
pom.xml  
src/main/java/com/ipn/mx/  
  ApiSimpleApplication.java  
  controller/TemperaturaController.java  
  service/TemperaturaService.java  
  dto/ConversionResponse.java  
  dto/ErrorResponse.java  
  exception/ValorFueraDeRangoException.java  
  exception/GlobalExceptionHandler.java  
src/main/resources/application.properties  
src/test/java/com/ipn/mx/service/TemperaturaServiceTest.java
```

El controlador solo recibe la petición y arma la respuesta; el servicio hace la aritmética y valida el rango; el manejador global convierte las excepciones en respuestas de error. Para compilar, probar y arrancar:

```
./mvnw package  
java -jar target/apisimple-0.0.1-SNAPSHOT.jar
```

2.4. Implementación

El DTO de respuesta es un **record**: sus valores se fijan al construirlo y no cambian, y como solo viaja de vuelta al cliente no necesita setters.

Código 1: DTO de respuesta

```
1 public record ConversionResponse(  
2     Double valorOriginal,  
3     String unidadOriginal,  
4     Double valorRespuesta,  
5     String unidadRespuesta  
6 ) {}
```

Cada endpoint sigue el mismo patrón: recibe `valor` con `@RequestParam`, llama al servicio y devuelve un `ResponseEntity.ok()` con el DTO [5].

Código 2: Endpoint de Celsius a Fahrenheit

```
1 @GetMapping("/convertir-celsius-a-fahrenheit")  
2 public ResponseEntity<ConversionResponse> celsiusAFahrenheit(  
3     @RequestParam(name = "valor") Double celsius  
4 ) {  
5     Double fahrenheit = servicio.celsiusAFahrenheit(celsius);  
6     return ResponseEntity.ok(new ConversionResponse(celsius, CELSIUS, fahrenheit,  
7         FAHRENHEIT));  
}
```

La validación del rango vive en el servicio, en un solo método que usan las seis conversiones, y el redondeo a dos decimales en otro.

Código 3: Validación del rango y redondeo en TemperaturaService

```
1 private void validar(double valor, double minimo, String unidad) {
2     if (Double.isNaN(valor) || Double.isInfinite(valor)) {
3         throw new ValorFueraDeRangoException("El valor debe ser un número finito.");
4     }
5     if (valor < minimo) {
6         throw new ValorFueraDeRangoException(
7             "%s %s está por debajo del cero absoluto (mínimo permitido: %s %s)."
8             .formatted(valor, unidad, minimo, unidad));
9     }
10 }
11
12 private double redondear(double valor) {
13     return Math.round(valor * 100.0) / 100.0 + 0.0;
14 }
```

Los errores se convierten en JSON con una clase `@RestControllerAdvice`, que captura la excepción antes de que llegue al cliente como una página de error de Spring [6].

Código 4: Dos de los tres manejadores de GlobalExceptionHandler

```
1 @ExceptionHandler(MethodArgumentTypeMismatchException.class)
2 public ResponseEntity<ErrorResponse> valorNoNumerico(
3     MethodArgumentTypeMismatchException ex, HttpServletRequest request
4 ) {
5     String mensaje = "El parámetro '%s' debe ser un número (con punto decimal); se
6     recibió '%s'."
7     .formatted(ex.getName(), ex.getValue());
8     return responder(HttpStatus.BAD_REQUEST, mensaje, request);
9 }
10
11 @ExceptionHandler(ValorFueraDeRangoException.class)
12 public ResponseEntity<ErrorResponse> fueraDeRango(
13     ValorFueraDeRangoException ex, HttpServletRequest request
14 ) {
15     return responder(HttpStatus.UNPROCESSABLE_CONTENT, ex.getMessage(), request);
16 }
```

El tercer manejador, para `MissingServletRequestParameterException`, responde 400 cuando falta valor.

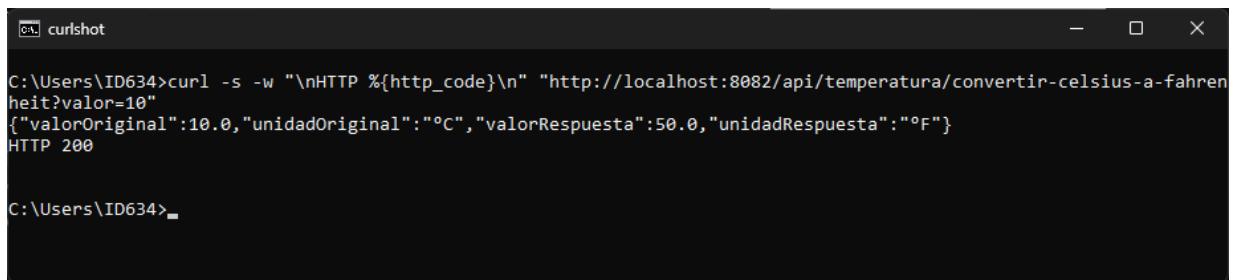
Además del arranque del contexto, hay cuatro pruebas unitarias de JUnit sobre el servicio: las seis fórmulas con valores conocidos, el cero absoluto exacto de cada escala (que debe aceptarse), un valor apenas por debajo de él (que debe rechazarse) y NaN e infinito. Las cinco pasan con `./mvnw package`.

2.5. Pruebas con curl

Todos los comandos se ejecutaron en una terminal de Windows contra el JAR corriendo en el puerto 8082. Se usan dos opciones de `curl` [7]: `-s` (*silent*) oculta la barra de progreso y `-w "\nHTTP %{http_code}\n"` imprime el código HTTP de la respuesta después del cuerpo. La URL va entre comillas porque contiene `?`, un carácter que algunos intérpretes de comandos interpretan por su cuenta. Para cada endpoint se muestra el comando, la captura de la terminal, la respuesta en JSON (con sangría para leerla mejor; la captura muestra el cuerpo tal cual llega) y la misma respuesta en texto plano.

2.5.1. Celsius a Fahrenheit

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-celsius-a-fahrenheit?valor=10"
```



```
C:\Users\ID634>curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-celsius-a-fahrenheit?valor=10"
{"valorOriginal":10.0,"unidadOriginal":"°C","valorRespuesta":50.0,"unidadRespuesta":"°F"}
HTTP 200

C:\Users\ID634>_
```

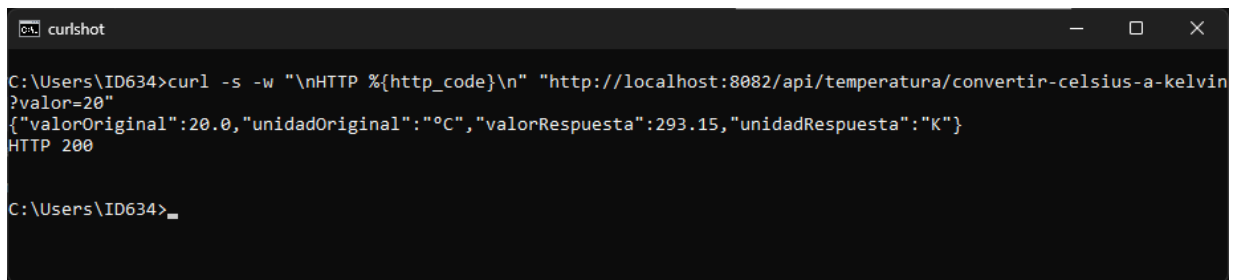
Figura 1: Celsius a Fahrenheit con valor=10

```
{ "valorOriginal": 10.0, "unidadOriginal": "°C", "valorRespuesta": 50.0, "unidadRespuesta": "°F" }
```

Texto plano: 10 °C equivalen a 50 °F (HTTP 200).

2.5.2. Celsius a Kelvin

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-celsius-a-kelvin?valor=20"
```



```
C:\Users\ID634>curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-celsius-a-kelvin?valor=20"
{"valorOriginal":20.0,"unidadOriginal":"°C","valorRespuesta":293.15,"unidadRespuesta":"K"}
HTTP 200

C:\Users\ID634>_
```

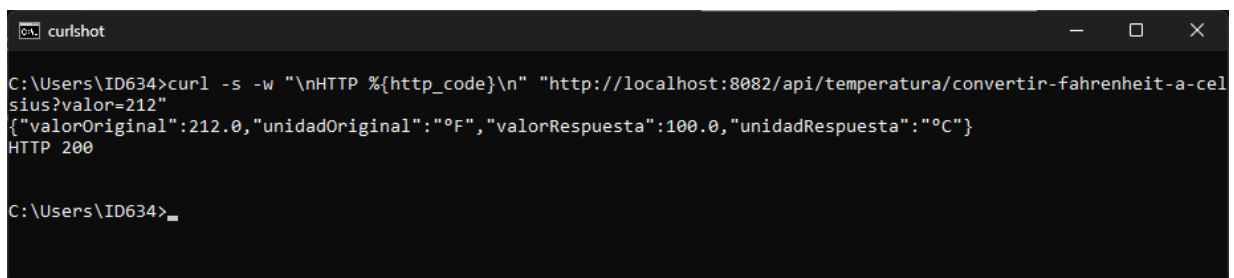
Figura 2: Celsius a Kelvin con valor=20

```
{ "valorOriginal": 20.0, "unidadOriginal": "°C", "valorRespuesta": 293.15, "unidadRespuesta": "K" }
```

Texto plano: 20 °C equivalen a 293.15 K (HTTP 200).

2.5.3. Fahrenheit a Celsius

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-fahrenheit-a-celsius?valor=212"
```



```
C:\Users\ID634>curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-fahrenheit-a-celsius?valor=212"
{"valorOriginal":212.0,"unidadOriginal":"°F","valorRespuesta":100.0,"unidadRespuesta":"°C"}
HTTP 200

C:\Users\ID634>_
```

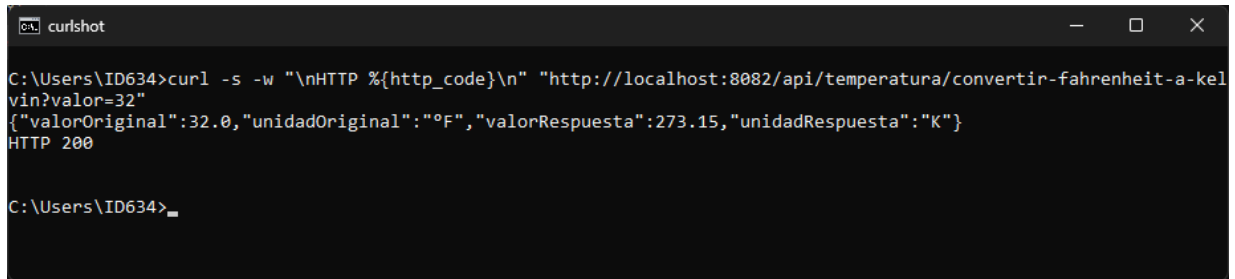
Figura 3: Fahrenheit a Celsius con valor=212

```
{ "valorOriginal": 212.0, "unidadOriginal": "°F", "valorRespuesta": 100.0, "unidadRespuesta": "°C" }
```

Texto plano: 212 °F equivalen a 100 °C, el punto de ebullición del agua (HTTP 200).

2.5.4. Fahrenheit a Kelvin

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-fahrenheit-a-kelvin?valor=32"
```



A screenshot of a terminal window titled 'curlshot'. The command entered is `curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-fahrenheit-a-kelvin?valor=32"`. The output displayed is `{"valorOriginal":32.0,"unidadOriginal":"°F","valorRespuesta":273.15,"unidadRespuesta":"K"} HTTP 200`. The prompt shows the user is in the directory `C:\Users\ID634`.

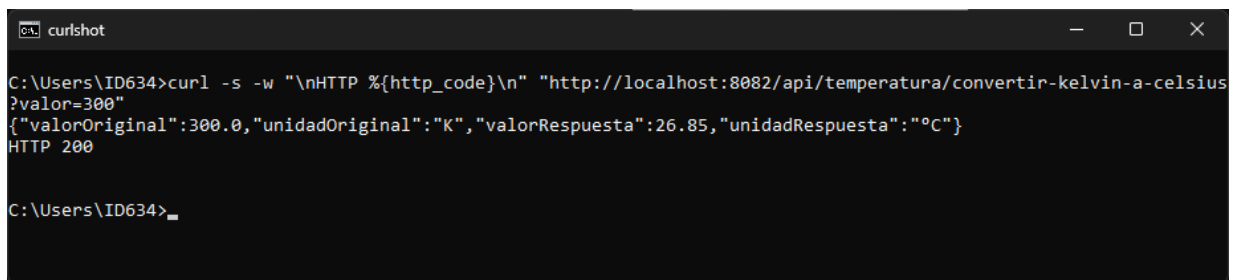
Figura 4: Fahrenheit a Kelvin con valor=32

```
{ "valorOriginal": 32.0, "unidadOriginal": "°F", "valorRespuesta": 273.15, "unidadRespuesta": "K" }
```

Texto plano: 32 °F equivalen a 273.15 K, el punto de congelación del agua (HTTP 200).

2.5.5. Kelvin a Celsius

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-kelvin-a-celsius?valor=300"
```



A screenshot of a terminal window titled 'curlshot'. The command entered is `curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-kelvin-a-celsius?valor=300"`. The output displayed is `{"valorOriginal":300.0,"unidadOriginal":"K","valorRespuesta":26.85,"unidadRespuesta":"°C"} HTTP 200`. The prompt shows the user is in the directory `C:\Users\ID634`.

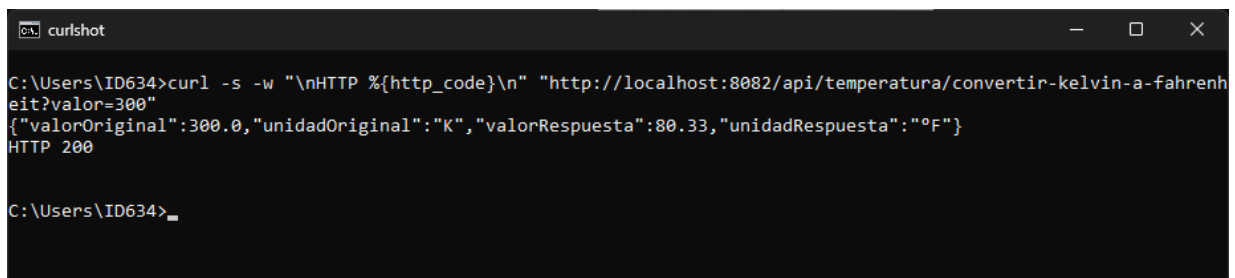
Figura 5: Kelvin a Celsius con valor=300

```
{ "valorOriginal": 300.0, "unidadOriginal": "K", "valorRespuesta": 26.85, "unidadRespuesta": "°C" }
```

Texto plano: 300 K equivalen a 26.85 °C (HTTP 200).

2.5.6. Kelvin a Fahrenheit

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-kelvin-a-fahrenheit?valor=300"
```



A screenshot of a terminal window titled 'curlshot'. The command entered is `curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-kelvin-a-fahrenheit?valor=300"`. The output displayed is `{"valorOriginal":300.0,"unidadOriginal":"K","valorRespuesta":80.33,"unidadRespuesta":"°F"} HTTP 200`. The prompt shows the user is in the directory `C:\Users\ID634`.

Figura 6: Kelvin a Fahrenheit con valor=300

```
{ "valorOriginal": 300.0, "unidadOriginal": "K", "valorRespuesta": 80.33, "unidadRespuesta": "°F" }
```

Texto plano: 300K equivalen a 80.33 °F (HTTP 200).

2.6. Validación de datos

Se probaron cinco entradas inválidas. En todos los casos la API responde con un JSON de error (el objeto de cuatro campos descrito en la tabla 2) y no con una traza de Java.

2.6.1. Valor no numérico

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-celsius-a-fahrenheit?valor=abc"
```

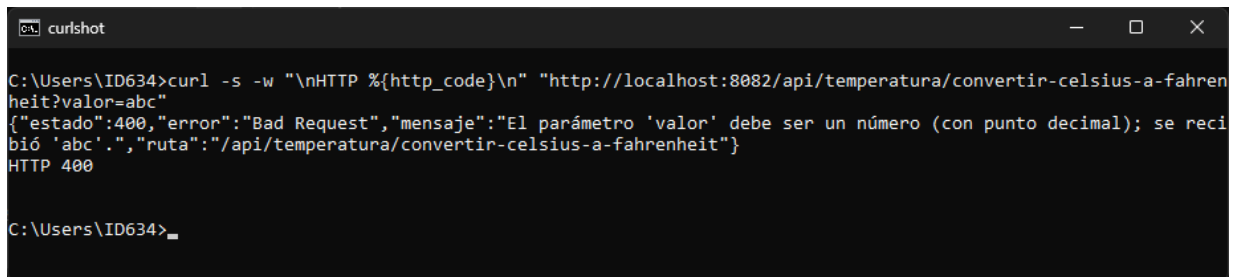


Figura 7: Valor no numérico: valor=abc

```
{ "estado": 400, "error": "Bad Request",  
  "mensaje": "El parámetro 'valor' debe ser un número (con punto decimal); se recibió 'abc'.",  
  "ruta": "/api/temperatura/convertir-celsius-a-fahrenheit" }
```

Texto plano: error 400: el parámetro valor debe ser un número; se recibió abc. Spring no puede convertir la cadena a Double y lanza `MethodArgumentTypeMismatchException`, que el manejador global convierte en este JSON. Lo mismo ocurre con una coma decimal (20,5).

2.6.2. Parámetro faltante

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-celsius-a-kelvin"
```

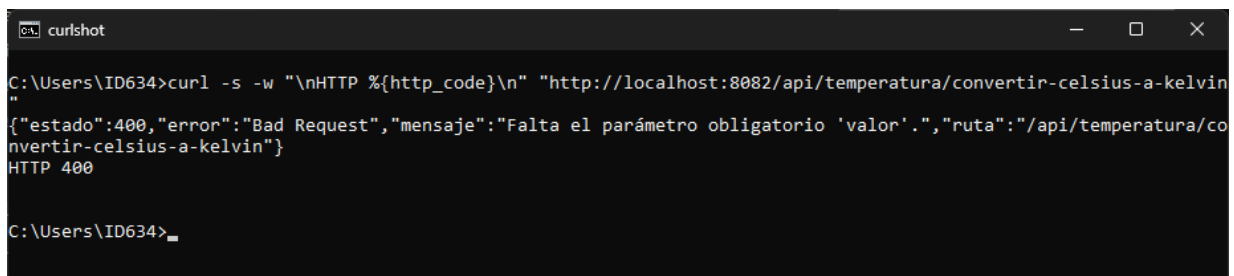


Figura 8: Petición sin el parámetro valor

```
{ "estado": 400, "error": "Bad Request", "mensaje": "Falta el parámetro obligatorio 'valor'.",  
  "ruta": "/api/temperatura/convertir-celsius-a-kelvin" }
```

Texto plano: error 400: falta el parámetro obligatorio valor.

2.6.3. Valor fuera de rango: por debajo del cero absoluto en Celsius

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-celsius-a-kelvin?valor=-300"
```

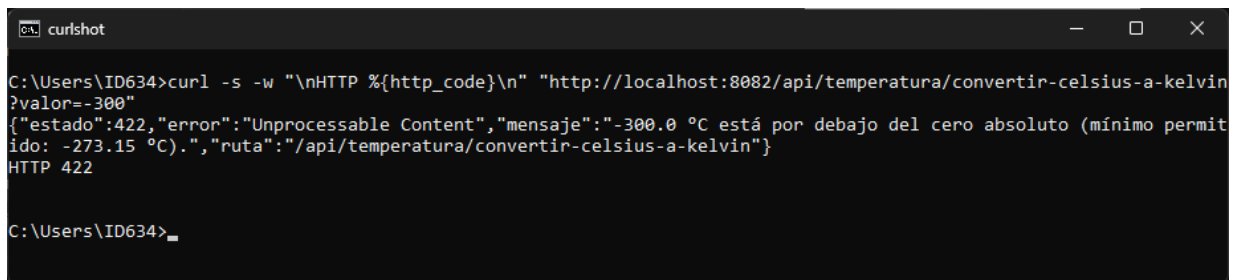


Figura 9: Valor por debajo del cero absoluto en Celsius

```
{ "estado": 422, "error": "Unprocessable Content",  
  "mensaje": "-300.0 °C está por debajo del cero absoluto (mínimo permitido: -273.15 °C).",  
  "ruta": "/api/temperatura/convertir-celsius-a-kelvin" }
```

Texto plano: error 422: -300°C está por debajo del cero absoluto; el mínimo permitido es $-273,15^{\circ}\text{C}$. El valor sí es un número, pero no existe físicamente, por eso se devuelve 422 y no 400.

2.6.4. Valor fuera de rango: Kelvin negativo

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-kelvin-a-celsius?valor=-5"
```

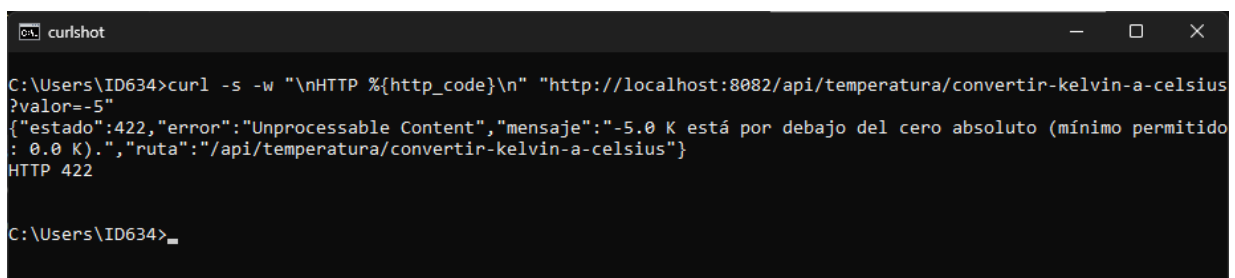


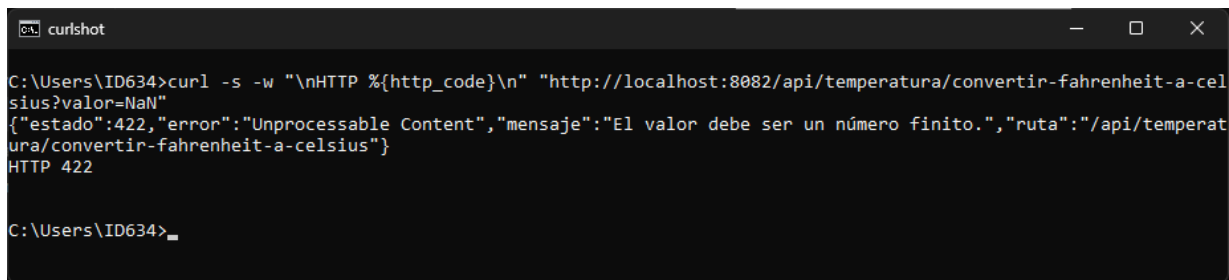
Figura 10: Kelvin negativo, por debajo de su mínimo

```
{ "estado": 422, "error": "Unprocessable Content",  
  "mensaje": "-5.0 K está por debajo del cero absoluto (mínimo permitido: 0.0 K).",  
  "ruta": "/api/temperatura/convertir-kelvin-a-celsius" }
```

Texto plano: error 422: -5 K está por debajo del cero absoluto; el mínimo permitido es 0 K .

2.6.5. Valor no finito: NaN

```
curl -s -w "\nHTTP %{http_code}\n" "http://localhost:8082/api/temperatura/convertir-fahrenheit-a-celsius?valor=NaN"
```



```
C:\Users\ID634>curl -s -w "%{http_code}\n" "http://localhost:8082/api/temperatura/convertir-fahrenheit-a-celsius?valor=NaN"
{"estado":422,"error":"Unprocessable Content","mensaje":"El valor debe ser un número finito.","ruta":"/api/temperatura/convertir-fahrenheit-a-celsius"}
HTTP 422

C:\Users\ID634>
```

Figura 11: Valor NaN

```
{ "estado": 422, "error": "Unprocessable Content",
  "mensaje": "El valor debe ser un número finito.",
  "ruta": "/api/temperatura/convertir-fahrenheit-a-celsius" }
```

Texto plano: error 422: el valor debe ser un número finito. NaN e Infinity sí se convierten a Double sin error [8], así que llegan hasta el servicio y ahí se rechazan.

3. Conclusiones

Los errores más típicos de esta práctica tienen que ver con los tipos de dato. Todo lo que llega por la URL es texto, y Spring lo convierte a `Double` antes de entrar al método. Si la conversión falla (abc, o 20,5 con coma decimal, que Java no acepta) el método nunca se ejecuta y el error se produce fuera de él, en `MethodArgumentTypeMismatchException`; por eso hay que capturarlo en un `@RestControllerAdvice` y no dentro del controlador. En sentido contrario, NaN e Infinity sí convierten sin problema a `Double`, de modo que un número “válido” para Java puede no serlo para el problema, y hay que rechazarlo a mano.

La diferencia entre `Double` y `double` también importa. El parámetro del controlador es el objeto `Double`, que puede ser `null`, y el servicio recibe el primitivo `double`. Eso solo es seguro porque el parámetro es obligatorio: si faltara, Spring responde 400 antes de llamar al método, y no se produce un `NullPointerException` al desempaquetar.

En la aritmética, dos errores son clásicos. El primero es la división entera: en Java `9 / 5` con enteros vale 1, no 1.8. En este proyecto la operación `celsius * 9 / 5` se evalúa de izquierda a derecha con un `double`, así que no ocurre, pero escribir `9 / 5 * celsius` con literales enteros sí lo produciría. El segundo es la precisión de la coma flotante: `0.1 + 0.2` da `0.30000000000000004`, y en esta API la conversión de 300 K a Fahrenheit da `80.33000000000004` antes de redondear. Por eso el servicio redondea a dos decimales, y además suma 0.0 al resultado para que un `-0,0` salga como `0.0`.

En cuanto a HTTP, la API separa el 400 del 422: 400 significa que la petición no se entiende (falta el parámetro o no es un número), y 422 significa que se entiende pero el valor no tiene sentido (por debajo del cero absoluto) [1]. Si el servicio lanzara su excepción y nadie la capturara, Spring respondería 500, y un error del cliente se reportaría como un fallo del servidor. Otros errores frecuentes al probar con `curl` son usar el puerto 8080 en vez del 8082 (la conexión se rechaza si nada escucha ahí), olvidar las comillas alrededor de la URL, escribir `curl` en PowerShell (donde es un alias de `Invoke-WebRequest` y hay que usar `curl.exe`) y una consola sin UTF-8, donde el símbolo de grados sale corrupto. Poner el valor en la ruta (`/valor/10`) en vez de en la query también da un 404, porque `@RequestParam` lee el query string y no un segmento de ruta.

Las seis conversiones y los cinco casos de error se comprobaron con `curl` sobre la aplicación en ejecución (figuras 1 a 11).

4. Referencias Bibliográficas

- [1] Roy T. Fielding, Mark Nottingham y Julian Reschke. *RFC 9110: HTTP Semantics*. 2022. URL: <https://www.rfc-editor.org/rfc/rfc9110.html> (visitado 19-09-2026).
- [2] Bureau International des Poids et Mesures. *The International System of Units (SI) Brochure*. URL: <https://www.bipm.org/en/publications/si-brochure> (visitado 19-09-2026).
- [3] Brian Goetz y Alex Buckley. *JEP 395: Records*. URL: <https://openjdk.org/jeps/395> (visitado 19-09-2026).
- [4] Broadcom Inc. *Spring Boot Reference Documentation*. URL: <https://docs.spring.io/spring-boot/reference/> (visitado 19-09-2026).

- [5] Broadcom Inc. *Spring Framework Reference: Web on Servlet Stack, @RequestParam*. URL: <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-methods/requestparam.html> (visitado 19-09-2026).
- [6] Broadcom Inc. *Spring Framework Reference: Web on Servlet Stack, @ExceptionHandler*. URL: <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-exceptionhandler.html> (visitado 19-09-2026).
- [7] Daniel Stenberg y contribuidores. *curl man page*. URL: <https://curl.se/docs/manpage.html> (visitado 19-09-2026).
- [8] Oracle Corporation. *Java SE 21 API: java.lang.Double*. URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Double.html> (visitado 19-09-2026).
- [9] Roy Thomas Fielding. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doct. University of California, Irvine, 2000. URL: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.